NAFLD fibrosis modeling handbook: from the train/test split onward

For the modeling teammate. It continues the EDA (steps 1 to 6, already done) and explains every action from step 7 to the final report: what to run, why, what to observe and how to write it up. Follow the steps in order. The code uses single quotes and expects the setup cell from the EDA handover document to have been run in the same Colab session.

0. Before you start

Run the setup cell from the EDA handover, then this check cell. If it passes, nothing is missing and nothing leaks.

```
from sklearn.model_selection import train_test_split,
RepeatedStratifiedKFold, StratifiedKFold, cross_validate, cross_val_predict
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.base import clone
from xgboost import XGBClassifier

assert len(feats) == 33 and len(num_feats) == 27 and len(bin_cols) == 5
banned = set(biopsy_cols) | {'Patient No.', 'n_panel', 'in_panel', 'FIB4'}
assert not (set(feats) & banned), 'a leakage column is in feats'
print(y.value_counts(normalize=True).round(3))   # about 0.638 / 0.362
```

Golden rules for the whole project:

- The test set is touched once, at the very end.
- Every preprocessing step lives inside the Pipeline, so it is fitted on training folds only.
- No biopsy column, FIB4, n_panel or in_panel is ever an input.
- Use SEED = 42 everywhere and report mean plus spread, never a single lucky number.
- Every model is compared with FIB-4 on the same patients.

1. How steps 1 to 6 feed the modeling

What the EDA found	What it means for modeling
All 605 patients already have fatty liver (steatosis grades 1 to 3 only)	The target is fibrosis, not fatty liver detection
Significant fibrosis is 36.2% positive (219 of 605)	Mild imbalance: use class weights, stratified splits, PR-AUC and recall. SMOTE is not needed
Biopsy columns and derived flags would reveal the answer	They are excluded. The inputs are the 33 columns in feats
15 lab columns are missing mostly because only sicker patients were tested	They are dropped, and no missing-value indicators are used in the main model
The extended lab panel marks a sicker subgroup (fibrosis about 25% without it, about 48% with it)	Stratify on target plus in_panel, and report results for each group separately
Mild redundancy (AST and ALT 0.84, BMI and Waist 0.76)	Fine for trees. Logistic regression uses regularization. Run the body-size ablation
Every single feature is modest (best single-feature AUC about 0.65)	Expect a modest combined result, roughly AUC 0.70 to 0.78
FIB-4 has AUC 0.703 and sensitivity only 0.50 at cutoff 1.3	This is the benchmark, and sensitivity is where a model can add the most
The provided AtRiskNASH flag is unreliable (44% mismatch)	It is not used as a target. The recomputed AtRiskNASH_calc is a note only

2. Experiments and models: what to build and why

Three prediction tasks use the same features, the same pipeline and the same model families.

Experiment	Target	Type	Positives or classes	Why
A (main)	SigFibrosis	Binary	219 of 605 (36.2%)	Clinically important threshold, best balance, benchmark exists

Experiment	Target	Type	Positives or classes	Why
B (secondary)	AdvFibrosis	Binary	108 (17.9%)	The most urgent clinical group. FIB-4 is stronger here (AUC 0.767), so it is a harder benchmark
C (secondary)	FibGroup (0 = F0-1, 1 = F2, 2 = F3-4)	3-class	386 / 111 / 108	Shows the model separates severity levels, not just yes or no. Cirrhosis alone has only 24 patients, so F3 and F4 are merged

If the team decides to run at-risk NASH instead of C, note that AtRiskNASH_calc differs from SigFibrosis in only 10 patients, so it would repeat experiment A. That is why C is recommended.

Three classification models are trained for each experiment:

Model	Why it is included
Logistic regression	Simple, linear, interpretable baseline. If it matches the others, the signal is mostly additive
Random forest	Captures interactions and non-linear effects, robust on small data, little tuning needed
XGBoost	Usually the strongest on tabular data. Shallow trees and a slow learning rate suit 484 training patients

Why not deep learning: 605 rows is far too few. Why not many more models: with this sample size, extra models add noise and invite overfitting the CV score. FIB-4 is the clinical comparison model, not a trained one.

3. Step 7: split, pipeline, baseline models and comparison

7.1 Hold out the test set

```
X_train, X_test, y_train, y_test, s_train, s_test = train_test_split(
    X, y, strata, test_size=0.2, stratify=strata, random_state=SEED)
print(X_train.shape, X_test.shape)                    # (484, 33) (121, 33)
print(y_train.mean().round(3), y_test.mean().round(3))  # both near 0.362
print(data.loc[X_test.index, 'in_panel'].mean().round(2))  # near 0.49
```

Why: every choice made while modeling slowly fits the model to the data you looked at. The test set is the only honest final score. strata is the target plus in_panel as text, so both the fibrosis rate and the panel mix stay the same in train and test. Do not look at X_test again until step 8.

7.2 Preprocessing pipeline

```
pre = ColumnTransformer([
    ('num', Pipeline([('imp', SimpleImputer(strategy='median')), ('sc',
StandardScaler())]), num_feats),
    ('bin', SimpleImputer(strategy='most_frequent'), bin_cols),
    ('smk', Pipeline([('imp', SimpleImputer(strategy='most_frequent')),
                      ('oh', OneHotEncoder(handle_unknown='ignore'))]),
[smoke]),
], sparse_threshold=0)   # dense output, needed later for SHAP
```

Group	Columns	Treatment	Why
Numeric	27	Median fill, then standard scaling	Median resists skewed labs (GGT, AST). Scaling matters for logistic regression
Binary	5	Most-frequent fill only	Already 0/1 (gender is 1/2)
Smoking	1	Most-frequent fill, then one-hot	Three categories, so 1, 2, 3 must not be read as a numeric scale

The output has about 35 columns (27 + 5 + 3 smoking dummies). No missing-value indicators, on purpose: the tests that were ordered selectively are already dropped.

7.3 The three models

```
spw = (y_train == 0).sum() / (y_train == 1).sum()   # about 1.77

def make_models(spw):
    return {
        'LogReg': LogisticRegression(max_iter=3000, class_weight='balanced'),
        'RandomForest': RandomForestClassifier(n_estimators=500,
min_samples_leaf=5,
                                               class_weight='balanced',
random_state=SEED, n_jobs=-1),
        'XGBoost': XGBClassifier(n_estimators=300, max_depth=3,
learning_rate=0.05, subsample=0.8,
                                 colsample_bytree=0.8, scale_pos_weight=spw,
                                 eval_metric='logloss', random_state=SEED),
    }
models = make_models(spw)
```

Class weighting replaces SMOTE: mistakes on fibrosis patients count about 1.8 times more. min_samples_leaf=5 and max_depth=3 limit overfitting.

7.4 Repeated cross-validation

```
def run_cv(models, X_tr, y_tr, strat, scoring=None):
    scoring = scoring or ['roc_auc', 'average_precision', 'recall',
'precision', 'f1']
    splits = list(RepeatedStratifiedKFold(n_splits=5, n_repeats=5,
random_state=SEED).split(X_tr, strat))
    rows = {}
    for name, m in models.items():
        r = cross_validate(Pipeline([('pre', pre), ('model', m)]), X_tr,
y_tr, cv=splits,
                           scoring=scoring, n_jobs=-1,
return_train_score=True)
        rows[name] = {s: '%.3f +/- %.3f' % (r['test_' + s].mean(), r['test_'
+ s].std()) for s in scoring}
        rows[name]['train ' + scoring[0]] = '%.3f' % r['train_' +
scoring[0]].mean()
    return pd.DataFrame(rows).T, splits

table, splits = run_cv(models, X_train, y_train, s_train)
table
```

Five folds repeated five times gives 25 fits per model. One 80/20 split of 484 patients is too noisy, so the mean and standard deviation across 25 fits are what you report.

Metrics, with their chance levels for this data:

Metric	Meaning	Chance level
ROC-AUC	Chance that a fibrosis patient scores higher than a non-fibrosis patient	0.50
PR-AUC (average_precision)	Precision-recall area, sensitive to the positive class	about 0.36
Recall (sensitivity)	Share of fibrosis cases found	depends on the cut
Precision	Share of flagged patients who truly have fibrosis	about 0.36
F1	Balance of precision and recall	depends on the cut

Recall, precision and F1 use the default 0.5 cut, so treat them as a first look. Step 8 tunes the cut. Always-predict-no accuracy would be 63.8%, which is why accuracy is not used.

7.5 Compare every model with FIB-4 on the same patients

FIB-4's 0.703 covers all patients, while a CV score covers different rows. So build out-of-fold predictions, where each training patient is scored by a model that never saw them, and compare on identical rows.

```
skf = list(StratifiedKFold(n_splits=5, shuffle=True,
random_state=SEED).split(X_train, s_train))
fib4 = data.loc[X_train.index, 'FIB4'].values
ok = ~np.isnan(fib4)            # one patient has no platelet count
yt = y_train.values

oof = {n: cross_val_predict(Pipeline([('pre', pre), ('model', m)]), X_train,
y_train,
                            cv=skf, method='predict_proba')[:, 1] for n, m in
models.items()}

rng = np.random.default_rng(SEED)
def paired_boot(y_, a, b, n=2000):
    d = []
    for _ in range(n):
        i = rng.integers(0, len(y_), len(y_))
        if y_[i].min() != y_[i].max():
            d.append(roc_auc_score(y_[i], a[i]) - roc_auc_score(y_[i], b[i]))
```

```
    return np.percentile(d, [2.5, 97.5]).round(3)

print('FIB-4 AUC:', round(roc_auc_score(yt[ok], fib4[ok]), 3))
for n, p in oof.items():
    print(n, 'AUC', round(roc_auc_score(yt[ok], p[ok]), 3), '| minus FIB-4,
95% CI', paired_boot(yt[ok], p[ok], fib4[ok]))
```

How to decide:

- If the interval for the difference excludes 0, the model beats FIB-4 by more than chance.
- If it includes 0, write comparable, with no clear gain in AUC.
- Pick the best model by mean CV AUC, then by PR-AUC. If two models are within one standard deviation, prefer the simpler or more interpretable one.
- Record the results in a table like this one:

Model	CV ROC-AUC	CV PR-AUC	Train AUC	OOF AUC vs FIB-4 (95% CI)
FIB-4 (benchmark)	0.703 on all patients	not computed	not applicable	reference
Logistic regression	fill in	fill in	fill in	fill in
Random forest	fill in	fill in	fill in	fill in
XGBoost	fill in	fill in	fill in	fill in

7.6 Two sanity checks

```
y_shuf = pd.Series(np.random.default_rng(SEED).permutation(y_train.values),
index=y_train.index)
r = cross_validate(Pipeline([('pre', pre), ('model', models['XGBoost'])]),
X_train, y_shuf, cv=skf, scoring='roc_auc')
print('Shuffled-label AUC:', round(r['test_score'].mean(), 3))   # must be
about 0.5
```

- If the shuffled-label AUC is clearly above 0.5, something in the pipeline leaks. Stop and find it.
- If the real CV AUC is above about 0.9, suspect leakage first: print feats and check for biopsy columns, FIB4, n_panel or in_panel.
- A train AUC far above the CV AUC (for example 0.99 vs 0.72) means overfitting. Reduce depth or raise min_samples_leaf.

7.7 What you should observe and report

- Which model has the highest mean AUC, and whether the gaps exceed the standard deviations.
- Whether logistic regression is close to the tree models (a mostly additive signal) or clearly behind (interactions matter).
- The gap between train and CV scores for each model.
- Whether the best model beats FIB-4's 0.703, with the interval from the paired bootstrap.

4. Step 8: tune, choose a threshold, explain, check subgroups

8.1 Tune the best model, lightly

```
from sklearn.model_selection import RandomizedSearchCV
pipe = Pipeline([('pre', pre), ('model', XGBClassifier(eval_metric='logloss',
scale_pos_weight=spw, random_state=SEED))])
grid = {'model__max_depth': [2, 3, 4], 'model__n_estimators': [100, 200, 300,
500],
        'model__learning_rate': [0.01, 0.03, 0.05, 0.1], 'model__subsample':
[0.7, 0.8, 1.0],
        'model__colsample_bytree': [0.6, 0.8, 1.0],
'model__min_child_weight': [1, 3, 5, 10]}
search = RandomizedSearchCV(pipe, grid, n_iter=40, scoring='roc_auc', cv=skf,
random_state=SEED, n_jobs=-1)
search.fit(X_train, y_train)
print(search.best_params_, round(search.best_score_, 3))
best = search.best_estimator_
```

Keep the search small: with 484 patients a large search overfits the CV score, and best_score_ is slightly optimistic. If tuning gains less than the CV standard deviation, say so. Use the model family that won in step 7 (swap the estimator if it was the forest or logistic regression).

8.2 Choose the decision threshold on training data only

```
p_best = cross_val_predict(clone(best), X_train, y_train, cv=skf,
method='predict_proba')[:, 1]
fpr, tpr, thr = roc_curve(yt, p_best)
i = np.argmax(tpr >= 0.80)
THRESH = float(thr[i])
print('threshold', round(THRESH, 3), 'sensitivity', round(tpr[i], 2),
'specificity', round(1 - fpr[i], 2))
```

Why: the default 0.5 is arbitrary, and FIB-4′s weakness is sensitivity (0.50 at cutoff 1.3 with specificity 0.78). The aim is higher sensitivity at similar specificity. Compare the specificity you get at 80% sensitivity with FIB-4′s 0.78 at 50%. Lock THRESH before touching the test set.

8.3 Explain with SHAP

```
import shap
Xt = best.named_steps['pre'].transform(X_train)
names = best.named_steps['pre'].get_feature_names_out()
sv = shap.TreeExplainer(best.named_steps['model']).shap_values(Xt)
shap.summary_plot(sv, Xt, feature_names=names)
shap.summary_plot(sv, Xt, feature_names=names, plot_type='bar')
```

Check whether the top features match the EDA and clinical knowledge: age, glucose, HbA1c, platelets (lower is riskier), GGT, AST, BMI and diabetes. Agreement builds trust. A surprising top feature calls for a leakage check. Correlated features share credit, so say associated with, never causes.

8.4 Check subgroups

```
def boot_auc_ci(y_, s_, n=1000):
    out = []
    for _ in range(n):
        i = rng.integers(0, len(y_), len(y_))
        if y_[i].min() != y_[i].max():
            out.append(roc_auc_score(y_[i], s_[i]))
    return np.percentile(out, [2.5, 97.5]).round(3)

grp = data.loc[X_train.index, 'in_panel'].values
for g in [0, 1]:
    m_ = grp == g
    print('in_panel =', g, '| n =', m_.sum(), '| model AUC',
round(roc_auc_score(yt[m_], p_best[m_]), 3), boot_auc_ci(yt[m_], p_best[m_]))
    mk = m_ & ok
    print('   FIB-4 AUC', round(roc_auc_score(yt[mk], fib4[mk]), 3))
    print('   sensitivity', round((p_best[m_][yt[m_] == 1] >= THRESH).mean(),
2),
          'specificity', round((p_best[m_][yt[m_] == 0] < THRESH).mean(), 2))
```

The panel group has about twice the fibrosis rate. A model that performs similarly in both groups is far more convincing. Optionally repeat by gender and diabetes status.

8.5 Final evaluation on the test set, once

```
final = clone(best).fit(X_train, y_train)
p_test = final.predict_proba(X_test)[:, 1]
yte = y_test.values
fib4_te = data.loc[X_test.index, 'FIB4'].values
ok_te = ~np.isnan(fib4_te)
print('Test AUC model:', round(roc_auc_score(yte, p_test), 3),
boot_auc_ci(yte, p_test))
print('Test AUC FIB-4 :', round(roc_auc_score(yte[ok_te], fib4_te[ok_te]),
3))
pred = p_test >= THRESH
print('Sensitivity', round(pred[yte == 1].mean(), 2), 'Specificity',
round((~pred[yte == 0]).mean(), 2))
```

With only 121 test patients the intervals will be wide, and that is worth saying. If the test score is much lower than the CV score, report both honestly and do not retune.

5. Step 9: the secondary experiments

Reuse the same features, pipeline, make_models and run_cv. Only the target and the split change.

Experiment B: advanced fibrosis (binary)

```
yB = data['AdvFibrosis']
stratB = yB.astype(str) + '_' + data['in_panel'].astype(str)
XtrB, XteB, ytrB, yteB, strB, steB = train_test_split(X, yB, stratB,
test_size=0.2, stratify=stratB, random_state=SEED)
sposB = (ytrB == 0).sum() / (ytrB == 1).sum()          # about 4.6
tableB, splitsB = run_cv(make_models(sposB), XtrB, ytrB, strB)
tableB
```

Only about 86 training positives, so expect larger standard deviations. PR-AUC chance is about 0.18. The benchmark is FIB-4's AUC of 0.767 (CI 0.718 to 0.814), and the comparison method is the same as in 7.5 with yB. Report whether the model beats it, and be cautious if the gain is small.

Experiment C: fibrosis group (3 classes)

```
yC = data['FibGroup']
stratC = yC.astype(str) + '_' + data['in_panel'].astype(str)
XtrC, XteC, ytrC, yteC, strC, steC = train_test_split(X, yC, stratC,
test_size=0.2, stratify=stratC, random_state=SEED)
modelsC = {
    'LogReg': LogisticRegression(max_iter=3000, class_weight='balanced'),
    'RandomForest': RandomForestClassifier(n_estimators=500,
min_samples_leaf=5, class_weight='balanced',
                                           random_state=SEED, n_jobs=-1),
    'XGBoost': XGBClassifier(objective='multi:softprob', n_estimators=300,
max_depth=3, learning_rate=0.05,
                             subsample=0.8, colsample_bytree=0.8,
eval_metric='mlogloss', random_state=SEED),
}
tableC, splitsC = run_cv(modelsC, XtrC, ytrC, strC, scoring=
['balanced_accuracy', 'f1_macro', 'roc_auc_ovr'])
tableC

from sklearn.metrics import confusion_matrix
skfC = list(StratifiedKFold(n_splits=5, shuffle=True,
random_state=SEED).split(XtrC, strC))
pC = cross_val_predict(Pipeline([('pre', pre), ('model',
modelsC['XGBoost'])]), XtrC, ytrC, cv=skfC, method='predict')
print(confusion_matrix(ytrC, pC))
```

For three classes use balanced accuracy (chance 0.333), macro-F1 and one-vs-rest AUC, plus the confusion matrix. Expect the middle class (F2) to be the hardest, with most errors against its neighbours. Errors between F0-1 and F3-4 are the serious ones. FIB-4 is not a multi-class tool, so the baseline is the majority class (63.8% accuracy, balanced accuracy 0.333). The XGBoost model here is unweighted, so mention that when comparing it with the weighted models.

Optional sensitivity runs (report briefly)

- All five body-size columns (add Height and Weight back): does anything change?
- Add the 15 dropped lab columns back, or turn on missing-value indicators: scores will probably rise. That is the referral effect, so do not present it as a better model.
- Exploratory model on the roughly 283 patients with a near-complete lab panel. Different and sicker population, so never compare it directly with the main model.

6. Step 10: app, report and presentation

- Save the final fitted pipeline with joblib, together with THRESH and the feature list.
- Streamlit app: inputs are the 33 features (leave blank for missing), output is a risk score and a high or low flag at THRESH, plus a clear note that it is a research tool and not a diagnosis.
- Report outline: problem and why the goal changed, data and cleaning decisions (steps 1 to 6), methods, results tables for A, B and C, FIB-4 comparison, SHAP, subgroup results, limitations, conclusion.
- Figures: target balance, missingness chart, top-feature boxplots, ROC curves of the models and FIB-4 together, SHAP summary, confusion matrix for C.
- Limitations to state: single dataset, 605 patients, one centre, selective testing (referral effect), biopsy as the reference standard, no external validation, wide intervals on the test set.

7. Interpretation guide

What you observe	What it means	What to write
CV AUC about 0.70 to 0.78, close to FIB-4	Routine labs carry limited extra signal	Comparable to FIB-4, with the paired interval
Model beats FIB-4 with an interval that excludes 0	Real gain	Quote the gain and interval
Large standard deviation across folds	Small data, unstable estimate	Report mean and spread, avoid strong claims
Logistic regression close to XGBoost	Signal is mostly additive	Prefer the simpler model
Train AUC far above CV AUC	Overfitting	Reduce depth, raise leaf size
Higher sensitivity than FIB-4 at similar specificity	Clinical gain	This is the headline result
Subgroup AUCs differ a lot	Referral effect or selection	Report both and say so
AUC above 0.9 or shuffled-label AUC above 0.5	Probable leakage	Stop and audit feats

8. Common errors and fixes

Problem	Fix
NameError for a variable	Colab restarted or cells ran out of order. Run the setup cell first
Imputer or scaler fitted before the split	Keep everything inside the Pipeline
Using y instead of the strata labels when splitting	Pass strata (or stratB, stratC) to the splitter
SHAP fails on sparse output	Keep sparse_threshold=0 in the ColumnTransformer
Looking at X_test during tuning	Re-do the split with a new seed only if truly necessary, and say so in the report
Column not found	Check the original spellings: Trombosit, Diyabetes Mellitus

9. Deliverables checklist

- Setup and check cells pass
- Split done and verified (484 / 121, about 36% positives in both)
- CV table for A (three models) with train scores
- Paired comparison with FIB-4, with intervals
- Shuffled-label check done
- Tuned model, locked threshold, SHAP plots, subgroup results
- One final test-set evaluation
- Experiments B and C tables
- Saved model, app and report